CS 241 DATA STRUCTURES AND ALGORITHMS

Lecture 7

Hash tables

Constant-time lookup, collisions and the load factor

Dr. Amara Nwosu
School of Computing, Northfield University

Tuesday 14 October 2026

Today

- Hashing
- Collisions
- Load factor and resizing
- Practice

LEARNING GOALS

1. Explain how a hash function maps keys to slots.
2. Compare chaining with open addressing.
3. Predict the cost of a lookup from the load factor.
4. Decide when a table should grow.

PART 1

Hashing

TODAY

- **Hashing**
- Collisions
- Load factor and resizing
- Practice

From keys to slots

DEFINITION Hash function

A hash function h maps every key in a universe U to one of m slots, $h\colon U \to \{0, 1, \ldots, m-1\}$. A hash table stores key k in slot $h(k)$.

EXAMPLE Division method

With $m = 11$ and $h(k) = k \bmod 11$, the key 47 goes to slot 3, because $47 = 4 \cdot 11 + 3$.

A good h spreads keys evenly and is cheap to compute; choose m prime so that patterns in the keys do not line up with the table size.

Separate chaining in Python

```python
class ChainedTable:
    def __init__(self, m=11):
        self.slots = [[] for _ in range(m)]

    def index(self, key):
        return hash(key) % len(self.slots)

    def put(self, key, value):
        chain = self.slots[self.index(key)]
        for i, (k, _) in enumerate(chain):
            if k == key:   # overwrite
                chain[i] = (key, value)
                return
        chain.append((key, value))
```

- Each slot holds a list of the pairs that hash there.
- `put` scans one chain, never the whole table.
- `get` walks the same chain and raises `KeyError` on a miss.
- Cost of a lookup: one hash plus the length of one chain.

PART 2

Collisions

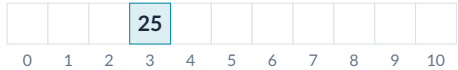
TODAY

- Hashing
- **Collisions**
- Load factor and resizing
- Practice

Linear probing, one insertion at a time

Insert 25, 47, 14, 36 into $m = 11$ slots with $h(k) = k \bmod 11$

1. $h(25) = 3$: slot 3 is free, store 25.
2. $h(47) = 3$: taken, probe 4, store 47.
3. $h(14) = 3$: probe 4, then 5, store 14.
4. $h(36) = 3$: three probes, store in 6.

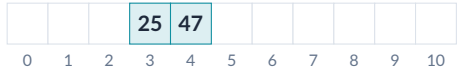


Keys that hash near a cluster join it: clusters grow faster the bigger they get.

Linear probing, one insertion at a time

Insert 25, 47, 14, 36 into $m = 11$ slots with $h(k) = k \bmod 11$

1. $h(25) = 3$: slot 3 is free, store 25.
2. $h(47) = 3$: taken, probe 4, store 47.
3. $h(14) = 3$: probe 4, then 5, store 14.
4. $h(36) = 3$: three probes, store in 6.

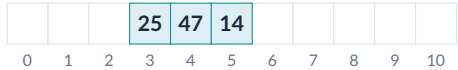


Keys that hash near a cluster join it: clusters grow faster the bigger they get.

Linear probing, one insertion at a time

Insert 25, 47, 14, 36 into $m = 11$ slots with $h(k) = k \bmod 11$

1. $h(25) = 3$: slot 3 is free, store 25.
2. $h(47) = 3$: taken, probe 4, store 47.
3. $h(14) = 3$: probe 4, then 5, store 14.
4. $h(36) = 3$: three probes, store in 6.

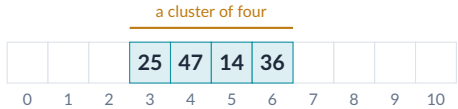


Keys that hash near a cluster join it: clusters grow faster the bigger they get.

Linear probing, one insertion at a time

Insert 25, 47, 14, 36 into $m = 11$ slots with $h(k) = k \bmod 11$

1. $h(25) = 3$: slot 3 is free, store 25.
2. $h(47) = 3$: taken, probe 4, store 47.
3. $h(14) = 3$: probe 4, then 5, store 14.
4. $h(36) = 3$: three probes, store in 6.



Keys that hash near a cluster join it: clusters grow faster the bigger they get.

How much does a collision cost?

THEOREM Expected probes, uniform hashing

In an open-addressing table with load factor $\alpha = n/m < 1$, an unsuccessful search makes at most $1/(1-\alpha)$ probes in expectation.

WATCH OUT Deleting under open addressing

Do not empty a slot on delete: a later search would stop there and miss keys stored past it. Mark the slot with a tombstone instead.

Chaining or open addressing?

	Separate chaining	Open addressing
Where colliding keys go	a list per slot	another slot of the array
Load factor α	may exceed 1	must stay below 1
Memory per key	one node and pointer	none beyond the array
Cache behaviour	poor, pointers	good, contiguous
Deletion	remove from list	needs tombstones

Python's `dict` uses open addressing; Java's `HashMap` uses chaining.

PART 3

Load factor and resizing

TODAY

- Hashing
- Collisions
- **Load factor and resizing**
- Practice

Worked example: when should the table grow?

A table with $m = 64$ slots holds $n = 48$ keys. How many probes does a failed lookup expect, and after doubling?

$$\alpha = \frac{n}{m} = \frac{48}{64} = 0.75$$

$$E[\text{probes}] \le \frac{1}{1-\alpha} = \frac{1}{0.25} = 4$$

$$\alpha' = \frac{48}{128} = 0.375$$

$$E'[\text{probes}] \le \frac{1}{0.625} = 1.6$$

EXAMPLE Rule of thumb

Doubling m when α passes 0.75 keeps lookups under four probes. Each key is rehashed into the new table, so one resize costs $O(n)$.

THEOREM Amortised cost

With doubling, n insertions cost $O(n)$ in total: $O(1)$ each, amortised.

PART 4

Practice

TODAY

- Hashing
- Collisions
- Load factor and resizing
- **Practice**

Quick question

Take a minute, then compare with your neighbour.

A linear-probing table has $m = 10$ and $\alpha = 0.9$. You insert one more key. What is the expected number of probes for the next failed lookup?

A About 1, the table is nearly empty

B About 2

C No bound: $\alpha = 1$, the table is full

D About 10

With ten keys in ten slots there is no empty slot to stop a probe sequence: the bound $1/(1 - \alpha)$ is undefined. Resize before this happens.

Quick question

Take a minute, then compare with your neighbour.

A linear-probing table has $m = 10$ and $\alpha = 0.9$. You insert one more key. What is the expected number of probes for the next failed lookup?

A About 1, the table is nearly empty

B About 2

C No bound: $\alpha = 1$, the table is full

D About 10

With ten keys in ten slots there is no empty slot to stop a probe sequence: the bound $1/(1 - \alpha)$ is undefined. Resize before this happens.

Summary

- A hash function turns a key into a slot in $O(1)$; collisions are unavoidable.
- Chaining keeps a list per slot; open addressing probes for another slot.
- The load factor $\alpha = n/m$ sets the cost: about $1/(1 - \alpha)$ probes.
- Grow the table, usually by doubling, before α gets close to 1.

DEFINITION Key term

Load factor α: the number of stored keys divided by the number of slots.

Next lecture

LECTURE 8 THURSDAY 16 OCTOBER

Binary search trees

Ordered keys, in-order traversal, and why a tree can answer questions a hash table cannot.

BEFORE THURSDAY

- Read chapter 12, sections 12.1 to 12.3
- Problem set 4 is due Friday at 17:00
- Office hours: Wednesday 14:00, room B214